

Larkos Training Loop and Inference Runner

Witold Warchol

June 2, 2026

Last updated: June 13, 2026

1 Overview

The Larkos system consists of two major components:

1. **TrainingLoop**: Full differentiable training with gradient computation, emotional state updates, and cognitive fusion
2. **LarkosRunner**: Inference-only forward pass that loads trained weights and runs without gradient accumulation

Both components share the same core modules: the `LarkosModel`, `_TemporalAttentionEncoder`, `_NeuronGraphReasoner`, `_FusionTransformerHead`, embedding networks, and text codec. The key difference is that `TrainingLoop` computes gradients and updates weights, while `LarkosRunner` only performs forward inference using frozen weights.

2 Core Mathematical Components

2.1 Fourier Encoding

The input $x \in \mathbb{R}^D$ is encoded using positional-style Fourier features:

$$\phi(x) = [\sin(2^0\pi x), \sin(2^1\pi x), \dots, \sin(2^{E-1}\pi x), \cos(2^0\pi x), \cos(2^1\pi x), \dots, \cos(2^{E-1}\pi x)]$$

where E is the number of encodings (`FOURIER_ENCODINGS`). The output has dimension $2 \cdot D \cdot E$.

Rationale: Fourier encodings help the model learn periodic and frequency-dependent patterns. They provide stable gradients compared to raw inputs and help with generalization.

2.2 Online Normalization

Two normalization strategies track running statistics:

2.2.1 OnlineMinMax

Per-dimension running min/max via exponential moving average (EMA):

$$m_t^{(i)} = m_{t-1}^{(i)} + \alpha(x_t^{(i)} - m_{t-1}^{(i)})$$

where $\alpha = 0.02$ (momentum). The normalized output is:

$$\tilde{x}^{(i)} = \frac{x^{(i)} - m}{M - m + \epsilon} \cdot 2 - 1$$

clamped to $[-3, 3]$.

Rationale: EMA-based min/max prevents early outliers from permanently warping the normalization range. The system gradually forgets extreme values and re-centers around the actual distribution.

2.2.2 OnlineMeanStd

Per-dimension running mean and variance via EMA:

$$\mu_t^{(i)} = \mu_{t-1}^{(i)} + \alpha(x_t^{(i)} - \mu_{t-1}^{(i)})$$

$$\sigma_t^{(i)} = (1 - \alpha) \left(\sigma_{t-1}^{(i)} + \alpha(x_t^{(i)} - \mu_{t-1}^{(i)})^2 \right)$$

Output normalization:

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu}{\sqrt{\sigma} + \epsilon_f}$$

where $\epsilon_f = 0.1$ (std floor).

Rationale: The std floor prevents near-constant dimensions from dividing by zero and blowing up. It keeps low-variance dimensions genuinely quiet rather than amplified by numerical instability.

2.3 Temporal Context Window and Encoder

The Fourier-encoded input is padded into a temporal window of size $w = \text{TEMPORAL_WINDOW} = 4$, forming a sequence:

$$X_{\text{seq}} = [x_{t-w+1}, x_{t-w+2}, \dots, x_t] \in \mathbb{R}^{w \times \text{FOURIER_OUT_DIM}}$$

Earlier positions are zero-padded. The sequence is passed through `_TemporalAttentionEncoder` before flattening:

$$X_{\text{enc}} = \text{TransformerEncoder}(\text{LayerNorm}(X_{\text{seq}} + \mathbf{P}_{\text{pos}})) \in \mathbb{R}^{w \times \text{FOURIER_OUT_DIM}}$$

$$x_{\text{temporal}} = \text{reshape}(X_{\text{enc}}) \in \mathbb{R}^{w \cdot \text{FOURIER_OUT_DIM}}$$

where $\mathbf{P}_{\text{pos}} \in \mathbb{R}^{w \times d}$ is a learned positional embedding. The encoder uses `TEMPORAL_NHEAD` = 4 heads, `TEMPORAL_LAYERS` = 2 layers, and `TEMPORAL_DIM_FF` = 64. The output shape is identical to a flat concatenation of the window frames, so the downstream `LarkosModel` contract is unchanged.

Rationale: The encoder gives the model an explicit temporal axis to attend over. The learned positional vector lets the model distinguish the current frame from older frames. Without the encoder, temporal position is only implicit in the slot offset of the flat vector.

Gradient note: `temporal_encoder` is listed in `DETACHED_LABELS` because chain-rule attenuation through many layers puts its gradient norm structurally orders of magnitude below the fusion head. Dead-module and oscillation checks still apply; only the head-vs-module balance ratio is skipped.

2.4 LarkosModel Internals

The `LarkosModel` maps the `MODEL_INPUT_DIM`-dim temporal observation to a `MAX_NEURONS`-dim neuron-output vector. It is a transformer encoder over a token sequence built by quantising each input dimension into a soft distribution over a learnable vocabulary, optionally prepended with a context CLS token.

2.4.1 NumericTokenizer

Each input dimension $x^{(i)}$ is encoded as a soft assignment over $V = \text{VOCAB_SIZE}$ learnable prototypes $\{p_j\}_{j=1}^V$:

$$w_{ij} = \frac{\exp(-(x^{(i)} - p_j)^2)}{\sum_{j'} \exp(-(x^{(i)} - p_{j'})^2)}$$

The per-feature token embedding is then a weighted sum over the embedding table $E \in \mathbb{R}^{V \times d_{\text{model}}}$:

$$t_i = \sum_{j=1}^V w_{ij} E_j \in \mathbb{R}^{d_{\text{model}}}$$

producing a sequence $(t_1, \dots, t_D)$ that the encoder attends over.

2.4.2 EmbeddingProjector (Context Stream)

When a context string s is supplied, it is encoded with a frozen pretrained sentence-transformer (MiniLM) and projected down to `PROJ_DIM`:

$$c = \text{LayerNorm}(W_{\text{proj}} \cdot \text{ST}(s)), \quad \text{ST}(\cdot) \text{ frozen.}$$

It is then projected up to d_{model} and prepended as a CLS-style token, so the encoder output at position 0 is the summary driving the linear head.

Per-epoch cache: The same s is consumed roughly $15\times$ per epoch (MC samples, main forward, MAML inner steps, verifier). `EmbeddingProjector` memoises the most-recent raw ST output keyed on s , so `ST(·)` fires once per epoch rather than once per forward pass.

Shared ST under deepcopy: A custom `__deepcopy__` returns a clone that re-uses the frozen ST module by reference. Without this, MAML’s per-epoch `copy`. `deepcopy(model)` would clone the $\sim 22\text{M}$ frozen MiniLM parameters on every inner update.

2.5 Text Bridge (Debug Readout)

The `_TextCodec` renders the C-side cognitive vector as English text. It is a *one-way readout*, never trained: previous attempts established that an 8-dim cognitive prefix cannot steer a frozen GPT-2, and a learned text objective fought L_{base} .

2.5.1 Source: Cognitive Vector, Not Head Output

The bridge consumes the C-side fused `_cog_raw` $\in \mathbb{R}^{\text{FUSION_DIM}}$ vector (pre-transformer) rather than the final $\mathbb{R}^{\text{MAX_NEURONS}}$ head output. The `_num_to_prefix` matrix is

$$W_{n \rightarrow p} \in \mathbb{R}^{(\text{N_PREFIX} \cdot \text{GPT2_HIDDEN}) \times \text{FUSION_DIM}}$$

with `requires_grad = False` (a frozen random projection persisted in the checkpoint).

2.5.2 wte-Stat Matching

A raw random projection lands far from the GPT-2 token-embedding distribution. After projecting and reshaping to $\text{N_PREFIX} \times \text{GPT2_HIDDEN}$, the prefix is rescaled to match E_{wte} per-tensor statistics:

$$\text{prefix} \leftarrow \frac{\text{prefix} - \mu_{\text{prefix}}}{\sigma_{\text{prefix}} + \epsilon} \cdot \sigma_{E_{\text{wte}}} + \mu_{E_{\text{wte}}}$$

so GPT-2 sees something whose moments resemble a real token embedding rather than out-of-distribution noise.

2.5.3 Anchored Decoding

The driver sentence (`anchor_text`) is tokenised, embedded through GPT-2’s E_{wte} , and concatenated *after* the prefix:

$$\text{inputs_embeds} = [\text{prefix}, E_{\text{wte}}(\text{anchor_ids})]$$

GPT-2 then continues from real text scaffolding while the prefix biases the continuation. When no anchor is provided, a single BOS embedding is used instead.

Degenerate-prefix guard: If $\sigma(\text{prefix}) < 10^{-3}$ the prefix carries no signal and would only push GPT-2 to repeat the same noise; `decode()` returns the empty string instead of calling `generate()`.

LM choice: GPT-2 is replaced with `distilgpt2` — identical tokenizer and `GPT2_HIDDEN = 768`, roughly half the weights and $\sim 2\times$ faster. The text is a debug readout, so the small quality drop is irrelevant.

3 Training Loop Architecture

3.1 Forward Pass

The forward pass (`_forward`) builds a fully differentiable computation graph:

$$\text{model_pred} = \text{LarkosModel}(x_{\text{temporal}}, \text{embed_ctx})$$

The `LarkosModel` outputs a vector that is split into an LLM embedding (`llm_embed`) and auxiliary dimensions.

3.1.1 Cross-Attention Path

The LLM embedding is gated and fused with text context:

$$\text{embed_gate} = \text{EmbedWeightNet}(x_{\text{norm}})$$

$$\text{llm_embed_g} = \text{llm_embed} \odot \text{embed_gate}$$

$$\text{llm_embed_ca} = \text{InputCrossAttention}(x_{\text{norm}}, \text{llm_embed_g})$$

where $\odot$ denotes element-wise multiplication.

Why Cross-Attention: Instead of concatenating the input and embedding, cross-attention treats the D -dimensional input as D separate 1-D query tokens and the E -dimensional embedding as key/value. This is far more expressive than simple concatenation and allows learned interaction.

3.1.2 Cognitive Fusion

The cross-attention embedding is detached (breaking the gradient graph at the C boundary) and passed to cognitive fusion:

$$\text{fused_cog_raw} = \text{cognitive_fuse}(\text{llm_embed_ca}, \text{neurons}, \text{mem_state}, \text{default_weights}, \dots)$$

Graph Breaking: The detach at the C boundary is intentional. The C-side fusion depends on backend neurons and memory state which are non-differentiable objects. The Python side cannot backprop into them. An auxiliary loss on the cross-attention embedding (`aux_proj`) provides the gradient signal for the modules before the C boundary.

3.1.3 Frozen Input Windows

Training targets are refreshed every `TARGET_FREEZE_INTERVAL` epochs. Between refreshes, the target and all fusion-head inputs are held constant:

$$\text{frozen_input} = (_cached_fused_cog \neq \text{None}) \wedge (_cached_driver \neq \text{None}) \wedge (_cached_graph_inputs \neq \text{None})$$

Three caches are maintained:

- `_cached_fused_cog`: the normalised C-side fused vector
- `_cached_driver`: the `llm_embed_ca` driver embedding
- `_cached_graph_inputs`: the raw GAT inputs (`node_features`, `adj_mask`, `edge_weight`) — pinned so the fusion head sees a consistent neuron snapshot; the GAT itself still runs `forward_from_inputs` in-graph on every step so its parameters receive gradient throughout the window

All three caches are invalidated together whenever the target is refreshed, so input and target always move in lockstep.

Rationale: Pinning the inputs and target on the same refresh schedule prevents the model from chasing a moving input toward a fixed target, which produces unstable loss. The GAT inputs are pinned at the C-side boundary, not at the GAT output, so the GAT parameters keep receiving gradient on every frozen epoch.

3.2 Neuron Graph Reasoner

Before the fusion head, the live neuron graph is processed by `_NeuronGraphReasoner`, producing per-neuron token embeddings:

$$\mathbf{H}_{\text{graph}} \in \mathbb{R}^{1 \times \text{MAX_NEURONS} \times d_{\text{gat}}}$$

Each node’s 8-dimensional feature vector ($D_{\text{node}} = 8$) consists of: state, output, layer one-hot (2 dims), tanh-squashed connection degree, per-neuron state velocity (Δstate), output magnitude, and mean outgoing edge weight. A learned per-neuron embedding $\mathbf{E}_{\text{neuron}} \in \mathbb{R}^{\text{MAX_NEURONS} \times d}$ is added so nodes with identical static features still produce distinct tokens.

Each `_GATLayer` computes multi-head attention scores using the GAT formulation:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}_{\text{src}}^\top W h_i + \mathbf{a}_{\text{dst}}^\top W h_j) \cdot (1 + \tanh(w_{ij}))$$

where w_{ij} is the C-side edge weight. Non-edges are masked to $-\infty$ before softmax. Self-loops are always present so an isolated neuron updates from itself. Two GAT layers (`GAT_HEADS = 2`, `GAT_LAYERS = 2`) with a residual + post-LayerNorm connect them.

The reasoner exposes two entry points:

- `build_graph_inputs(neurons)`: constructs node features, adjacency mask, and edge weight matrix from the live neurons dict. Also advances the `_prev_states` buffer for state-velocity computation.
- `forward_from_inputs(nf, am, ew)`: runs the in-graph projection and GAT layers on pre-built features. Used by the training loop so the GAT input can be cached while the GAT itself remains in-graph.

3.3 Fusion Transformer Head

The `_FusionTransformerHead` attends over a mixed token sequence of length `MAX_NEURONS+3`:

- `[0 : MAX_NEURONS)` — per-neuron tokens from `_NeuronGraphReasoner`
- `[MAX_NEURONS]` — `band_q` token (LLM query band, `BAND_Q = 32` dims projected to d_{model})
- `[MAX_NEURONS + 1]` — `band_m` token (memory band, `BAND_M = 32` dims projected to d_{model})
- `[MAX_NEURONS + 2]` — driver token (`llm_embed_ca`, `INTERNAL_DIM` dims projected to d_{model})

Token-type embeddings (4 types: graph / q / m / driver) let the encoder distinguish token kinds without individual positional indices for the 128 graph tokens:

$$\text{seq} = [\mathbf{H}_{\text{graph}} + \mathbf{e}_{\text{graph}}, \mathbf{q}_{\text{tok}} + \mathbf{e}_q, \mathbf{m}_{\text{tok}} + \mathbf{e}_m, \mathbf{d}_{\text{tok}} + \mathbf{e}_d]$$

A 3-layer encoder ($d_{\text{model}} = 64$, 4 heads, $d_{\text{ff}} = 128$) attends over the sequence. Output is pooled by a *learned-query attention pool*:

$$\text{scores} = \frac{\text{enc} \cdot \mathbf{q}_{\text{pool}}}{\sqrt{d_{\text{model}}}} \quad , \quad \text{pooled} = \sum_i \text{softmax}(\text{scores})_i \cdot \text{enc}_i$$

$$\text{output} = \text{LinearOut}(\text{pooled}) \in \mathbb{R}^{\text{MAX_NEURONS}}$$

Why learned-query pooling: A softmax over the 131-token sequence lets the pooling step learn to up-weight informative tokens rather than giving every token equal voice.

Frozen-input: In-graph dropout and noise are skipped on a frozen-window epoch so a pinned input is not regularised against a pinned target.

3.4 MAML Inner Loop

The outer-loss term (L_{outer}) is computed against a *fast clone* adapted to the current task with a few inner SGD steps. The clone is built once, at `TrainingLoop` initialisation, via `copy.deepcopy(self.model)` — the `EmbeddingProjector.__deepcopy__` override ensures the $\sim 22\text{M}$ frozen ST parameters are shared, not duplicated.

Each epoch refreshes only the trainable parameters in place:

$$\forall p_{\text{main}}, p_{\text{fast}} : p_{\text{main}}.\text{requires_grad} \Rightarrow p_{\text{fast}}.\text{data} \leftarrow p_{\text{main}}.\text{data}$$

then runs `MAML_INNER_STEPS` vanilla SGD steps on $L = \text{MSE}(\text{fast}(x, c), \text{target})$ with `lr = MAML_INNER_LR`. The adapted clone is returned and a single forward pass produces `maml_pred`.

MAML detach: `x_temporal` is detached before the inner loop. MAML’s inner `loss.backward()` would otherwise walk back through the shared temporal encoder graph and free its saved

intermediates before the outer `_backward` can use them. Detaching is also semantically correct: MAML fast-adapts the model, not the input pipeline. The temporal encoder still receives gradient via `model_pred` in the outer loss.

Why a persistent fast clone: Re-running `copy.deepcopy` inside every inner update traverses the entire model tree (including the frozen MiniLM) per epoch. The persistent clone reduces the per-epoch cost to the in-place copy of the small trainable subset.

3.5 Loss Computation

The loss is a weighted blend of four components:

$$L_{\text{total}} = \lambda_1 L_{\text{outer}} + \lambda_2 L_{\text{base}} + \lambda_3 L_{\text{pred}} + \lambda_4 L_{\text{aux}}$$

where:

$$L_{\text{outer}} = \text{SmoothL1}(\text{maml_pred}, \text{target})$$

$$L_{\text{base}} = \text{SmoothL1}(\text{fused}, \text{target})$$

$$L_{\text{pred}} = \text{SmoothL1}(\text{model_pred}, \text{target})$$

$$L_{\text{aux}} = \text{SmoothL1}(\text{aux_proj}(\text{llm_embed_ca}), \text{target})$$

The weights are:

$$\lambda_1 = \text{mc_blend}$$

$$\lambda_2 = (1 - \text{mc_blend}) \cdot 0.4$$

$$\lambda_3 = (1 - \text{mc_blend}) \cdot 0.3$$

$$\lambda_4 = 0.2$$

where `mc_blend` $\in [0.1, 0.5]$ is derived from Monte Carlo variance:

$$\text{mc_blend} = \max(0.5 - \sigma_{\text{MC}}^2 \cdot 0.5, 0.1)$$

Adaptive Blending: High MC variance (model uncertainty) increases the weight on the outer (MAML) loss, which is more stable. Low variance decreases it. The system adapts loss weights based on model confidence.

SmoothL1 Loss: Uses $\beta = 0.1$, giving L_2 near zero and L_1 far from zero. This prevents saturation unlike Cosine+L1 which fought itself.

Auxiliary Loss: The aux path carries gradient signal to cross-attention, embed-weight-net, and text-proj, which sit before the non-differentiable C boundary.

3.6 Gradient Balancing

The `embed_weight_net` branch is often weak. Balancing rescales weak gradients:

$$\text{scale} = \min \left(\frac{\text{norm}_{\text{FT}}}{10 \cdot \text{norm}_{\text{embed}}}, 5 \right)$$

where norms are EMA-smoothed to avoid per-epoch pulses.

$$p.\text{grad} \leftarrow p.\text{grad} \cdot \text{scale}$$

All gradients are then clipped globally:

$$\|g\|_2 \leq 1.0$$

Rationale: Prevents any single module from exploding and ensures balanced learning across branches.

3.7 Monte Carlo Sampling

During forward, the model runs in training mode and takes T stochastic samples to estimate output variance:

$$\text{samples} = [\text{model}(x + \epsilon_1), \dots, \text{model}(x + \epsilon_T)]$$

where $\epsilon_i \sim \mathcal{N}(0, 0.05^2) \cdot r_{\text{explore}}$ if exploration is above threshold, else zeros (dropout only).

$$\sigma_{\text{MC}}^2 = \text{Var}(\text{samples})$$

Conditional sample count: T depends on whether the run is actively exploring:

$$T = \begin{cases} \text{MC_DROPOUT_T} & \text{if } r_{\text{explore}} > \text{EXPLORE_THRESHOLD} \\ \max(\lfloor \text{MC_DROPOUT_T}/3 \rfloor, 2) & \text{otherwise} \end{cases}$$

When the only variance source is dropout, a small T captures the signal and `mc_blend` EMA smooths the rest. The full T fires only when Gaussian exploration noise is actually being injected, where the variance estimate has to integrate over noisy inputs and needs more samples to be meaningful. This cuts MC cost roughly $3\times$ on quiet epochs.

Rationale: Uncertainty estimates inform loss weighting. Without dropout, variance collapses and weights become uniform. Gaussian noise at high exploration keeps estimates meaningful.

3.8 Frozen Skip Logic

On a frozen (input, target) window, once the loss falls below `_frozen_skip_floor` (≈ 0.15), the optimizer step is skipped:

$$\text{skip} = (\text{in_frozen_window}) \wedge (L_{\text{loss}} < 0.15)$$

Rationale: Continuing to step a memorized pair drives gradients to $\sim 10^{-4}$ and wastes progress. Skipping prevents the sawtooth artifact in loss logs.

4 Emotional and Identity Updates

After each backward pass, side-effects update the C-side backend:

4.1 Novelty Derivation

From loss history:

$$\text{novelty} = \min\left(\frac{|L_{\text{current}} - \bar{L}|}{\bar{L} + 10^{-8}}, 1.0\right)$$

where $\bar{L}$ is the mean of the loss history. This is blended with the backend’s reflection:

$$\text{novelty} = \max(\text{novelty}, \text{reflect_novelty}, \text{reflect_drift})$$

4.2 Satisfaction and Emotion

Satisfaction combines low loss and improving loss:

$$\text{sat} = \min(\max(1.0 - L, 0) + \max(L_{\text{prev}} - L, 0), 1.0)$$

Emotion type and strength are picked based on loss delta and novelty:

$$\text{type} = \begin{cases} \text{SURPRISE} & \text{if novelty} > 0.7 \\ \text{LOVE} & \text{if } L_{\text{prev}} - L > 0.03 \\ \text{HATE} & \text{if } L - L_{\text{prev}} > 0.03 \\ \text{SURPRISE} & \text{otherwise (weak)} \end{cases}$$

$$\text{strength} = \begin{cases} \min(\text{novelty} \cdot 0.6, 0.6) & \text{type} = \text{SURPRISE} \\ \min(\Delta L \cdot 3.0, 0.6) & \text{type} = \text{LOVE/HATE} \\ 0.1 & \text{deadband} \end{cases}$$

Deadband: The 0.03 threshold prevents small wiggles from triggering extreme emotions. The 0.1 deadband emotion keeps arousal decaying instead of being pinned high.

5 Inference: LarkosRunner

5.1 Initialization and Loading

LarkosRunner reconstructs all forward-path modules in the same order as TrainingLoop so `load_checkpoint` can restore onto matching attribute names without special-casing:

`temporal_encoder, graph_reasoner, fusion_transformer, embed_weight_net, cross_attn, text_pr`

The C-side backend is reloaded with saved memory and network states:

```
backend.load_memory(), backend.load_network_states()
```

Rationale: Without the trained backend state, `cognitive_fuse` would see empty neurons and memory, and the readout would not reflect training dynamics. Both halves (PyTorch weights + C-side state) are needed together; neither is usable alone.

5.2 Backend Advance

Before each `step()` call reads neurons, memory, or input tensor, `_advance_backend(alpha)` runs the same C-side updates as the training loop’s pre-forward block:

```
run_decision_path, update_context, process_neurons, update_neuron_states, update_attractor,
```

Without this, successive `step()` calls in a multi-step inference loop read identical C-side state and return identical output.

Parameter defaults: `alpha` is derived from the live backend context via `derive_alpha_from_context` and `derive_alpha_from_params`, matching the training loop’s per-epoch derivation. `mem_weight_ratio` defaults to the band midpoint $\text{BASE} + 0.5 \times \text{RANGE}$, which sits inside the envelope the model was trained against. Both can be overridden explicitly.

5.3 Forward Inference

The runner’s `step` method mirrors `TrainingLoop._forward` but without:

- MC probes (no uncertainty sampling; frozen dropout)
- MAML inner loop (single forward pass only)
- Auxiliary loss computation (output path only)

The temporal encoder and graph reasoner are included in the forward path:

$$X_{\text{enc}} = \text{temporal_encoder}(X_{\text{seq}}) \quad , \quad \mathbf{H}_{\text{graph}} = \text{graph_reasoner}(\text{neurons})$$

The fused cognitive vector is split into bands before passing to the head:

```
band_q, band_m = split_band_q_m(normalize(fused_cog_raw))
```

All computation runs with `@torch.no_grad()`, preventing gradient accumulation and saving memory.

5.4 Input and Frozen Window Handling

Since inference has no training schedule, the frozen-window logic is disabled:

```
frozen_input = False
```

This mirrors the behavior of a target-refresh epoch in training.

5.5 Output

The runner returns a dictionary with:

- **fused**: The final neuron prediction (NumPy array)
- **model_pred**: Raw LarkosModel output
- **text_input**: The driver sentence
- **text_output**: Debug-readout text. Decoded by the text bridge (§ Text Bridge) from the C-side **fused_cog_raw** (FUSION_DIM) — *not* the smaller MAX_NEURONS head output — and anchored on **_current_text_input** when one is available, so GPT-2 has real text to continue from.

6 Key Design Decisions

6.1 Why Detach at the C Boundary

The C-side cognitive fusion reads non-differentiable backend objects (neurons, memory, network history). Python cannot build a computation graph through C code. Detaching prevents error and forces the gradient signal through the auxiliary loss instead.

6.2 Why Frozen Input Windows

Updating the target every epoch while the fusion-head inputs drift (due to neuron, memory, and driver changes) causes the loss to thrash. Pinning all inputs and the target on the same refresh schedule prevents chasing a moving target. The GAT inputs are pinned at the C-side boundary so the GAT parameters keep receiving gradient on every frozen epoch.

6.3 Why Separate Projections for Context Tokens

Each context token (band_q, band_m, driver) carries different information. Separate **nn.Linear** projections allow each stream to learn its own mapping into the shared d_{model} space and let attention determine their balance against the 128 GAT graph tokens.

6.4 Why SmoothL1 Loss

SmoothL1 with $\beta = 0.1$ avoids saturation of cosine losses and the fighting behavior of Cosine+L1. It is stable and well-behaved for vector regression.

6.5 Why Auxiliary Loss

The cross-attention, embed-weight-net, and text-proj networks sit before the C boundary. Without a separate loss on their outputs, they would receive zero gradient (the path to target is broken). The auxiliary loss regresses the cross-attention embedding to the target dimension and provides the needed signal.

7 Scheduler and Optimization

Training uses:

- **Optimizer:** AdamW with $\text{lr} = \text{BASE_LR}$, $\text{weight_decay} = 10^{-4}$, $\text{eps} = 10^{-6}$
- **Scheduler:** CosineAnnealingLR with $T_{\text{max}} = \text{COSINE_T_MAX}$, $\eta_{\text{min}} = 10^{-5}$
- **EMA:** Exponential moving average of main model weights (used for validation and inference)

Why AdamW, not Adam: Adam folds `weight_decay` into the gradient before adaptive scaling, which under-regularises; AdamW applies it directly to the weights and is the correct interpretation of `weight_decay = 10^{-4}`.

The optimizer covers all trainable modules: `model`, `fusion_transformer`, `graph_reasoner`, `temporal_encoder`, `embed_weight_net`, `cross_attn`, and `text_proj`.

The scheduler steps after each backward pass, ensuring a smooth cosine decay over the epoch range.

8 Conclusion

The Larkos training loop and runner form a unified system:

- **Training:** Full differentiable forward pass, multi-component loss, frozen windows, emotional state management
- **Inference:** Fast forward-only pass, loaded weights and backend state, no gradient overhead

Both share core modules and math but are optimized for their respective tasks. The design prioritizes stability (frozen windows, gradient balancing, adaptive loss weights), interpretability (emotional and identity updates), and efficiency (EMA, frozen sampling).